sEMG Keystroke Prediction

Zhengwen Tang
Univeristy of California, Los Angeles
zhengwen@ucla.edu

Xiaojin Zuo
Univeristy of California, Los Angeles
zuo0220@g.ucla.edu

Dibin Wu
Univeristy of California, Los Angeles
grs27@g.ucla.edu

Abstract

Surface EMG keystroke decoding is a challenging sequence modeling problem due to the presence of noisy signals and complex temporal dependencies. This project builds on the emg2qwerty dataset from Meta Reality Labs and the TDS_Conv_CTC baseline model provided by the UCLA ECE 147 course staff. The goal of this work is to explore neural network architectures beyond the baseline CNN framework to develop a more robust model for EMG-to-keystroke decoding. We evaluate several architectures, including recurrent models (Vanilla RNN, GRU, and LSTM), Residual + RNN, and Residual + RNN + Attention. All models are evaluated using Character Error Rate (CER). Experimental results show that the Residual TDS + RNN architecture performs best, reducing the test CER from 22.41 to 13.29, corresponding to approximately a 0.4 relative improvement over the baseline. This demonstrates the effectiveness of integrating residual of the baseline model with recurrent modeling for this task.

1 Introduction

This project is based on the emg2qwerty dataset released by Meta Reality Labs and utilizes the TDS-Conv-CTC baseline model provided for the UCLA ECE C147/C247 course. We address the following question: Can the performance of keystroke prediction be enhanced by introducing a more powerful sequence modeling module after the CNN-based baseline model? The emg2qwerty dataset comprises synchronously acquired surface electromyography (sEMG) signals from the wrist paired with their corresponding input text. Employing a CTC loss function [2], the baseline model enables the prediction of character sequences directly from continuous EMG input signals. We evaluate the performance of various architectures based on their Character Error Rate (CER) to identify the optimal architecture for practical application.

Based on this premise, we elected to retain the original convolutional front-end architecture while investigating whether the introduction of a sequence modeling module following the convolutional layers could effectively reduce the CER. We compared the original TDS-Conv-CTC baseline model against several extended models, specifically TDS+LSTM, Residual TDS+LSTM, and Residual TDS+LSTM augmented with a local attention mechanism. The primary objective across all these models is to optimize further the capacity for modeling temporal contextual information following the convolutional layers, while fully preserving the feature representations generated by the baseline model.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

2 Methods

2.1 Architecture

All models in this project are implemented based on the baseline architecture of emg2qwerty. The approach involves retaining the TDS frontend while modifying the sequence modeling backend.

2.1.1 TDS + Recurrent Model

To enhance temporal modeling capabilities, we first introduced recurrent layers following the convolutional encoder. We evaluated three recurrent architectures: Vanilla RNN, GRU, and LSTM. In this design, the convolutional front-end extracts local EMG features, while the recurrent layers model long-range dependencies within the sequence. These architectures enabled us to compare the effectiveness of different recurrent mechanisms for EMG-based keystroke decoding.

2.1.2 Residual TDS + LSTM

Based on preliminary experimental results, the LSTM variant demonstrated superior performance among all recurrent model variations. Consequently, to further extend this architecture, and inspired by the residual learning framework, a residual fusion structure was introduced. In this model, the output of the convolutional encoder is processed via two parallel paths: one path leads directly from the convolutional encoder to the classifier, while the other is a recurrent path in which features pass through an LSTM layer prior to classification. Subsequently, the outputs from these two paths are fused. This residual structure enables the model to simultaneously preserve the strong local features extracted by the convolutional encoder and integrate the global temporal contextual information captured by the LSTM.

2.1.3 Residual TDS + LSTM with Local Attention

The final model architecture further extends the residual LSTM model by incorporating a local cross-attention mechanism. In our implementation, this attention module takes the recurrent features produced by the first LSTM layer as one input stream, and the original TDS encoder features as the other. This enables the model to leverage locally aligned convolutional features to refine the recurrent representations. Consequently, this design bears a closer resemblance to local cross-attention than to the standard self-attention mechanism, as the attention interactions occur between two distinct feature streams rather than being confined within a single sequential representation. This attention mechanism is applied within fixed-size temporal windows, aiming to manage computational overhead while preserving locality. Subsequently, the fused representations are passed to a second recurrent module for further feature extraction.

2.2 Data Processing

This project utilizes a single-subject dataset from Subject 89335547, which has been partitioned into training, validation, and test sets. During the preprocessing phase, the raw EMG signals are first converted into tensors using ToTensor [3]. To enhance robustness against sensor variability and temporal misalignment, RandomBandRotation and TemporalAlignmentJitter (with a maximum offset of 120) [3] are applied during the training phase. Subsequently, the one-dimensional EMG signals are transformed into an STFT-based log spectrogram representation [1, 3] to extract frequency-domain information. To mitigate overfitting within the single-subject dataset, SpecAugment is applied by randomly masking blocks of data across both the time and frequency dimensions [1, 3]. During the validation and testing phases, all stochastic augmentation operations are disabled, retaining only the fundamental tensor conversion and spectrogram extraction steps [3].

2.3 Training Setup

All models were trained using the Connectionist Temporal Classification (CTC) framework in conjunction with a greedy decoding algorithm [2]. The Adam optimizer was selected as the optimization algorithm for the models. The initial learning rate was set to 1e-3 and was utilized in conjunction with a learning rate scheduler. Model performance was evaluated using the Character Error Rate (CER), which measures the edit distance between the predicted character sequence and the ground-truth text.

To efficiently explore the model architecture space, we first conducted a preliminary screening phase consisting of 40 training epochs. The models demonstrating the best performance were then selected for extended training, running for up to 150 epochs. Some LSTM variants also use bidirectional recurrence. All models employ a similar set of training hyperparameters to ensure a fair comparison across different architectures. The batch size is set to 32, and the input window length is 8,000 time steps, with padding of [1800, 200]. For recurrent architectures, the hidden layer size matches the feature dimension generated by the TDS encoder. In residual architectures, a dropout rate of 0.3 is applied to enhance generalization capabilities. For recurrent models, we experimented with 1–4 recurrent layers depending on the architecture.

3 Results

We first compare models within each family and then summarize the representative configurations across families.

3.1 Baseline

Model	**40 epoch test CER**	**150 epoch test CER**
Official TDS Baseline	23.99	22.41

Extending the official model from 40 to 150 epochs improves test CER only slightly, from 23.99 to 22.41. This makes it unlikely that the larger gains reported below come from training longer alone. The baseline provides the reference for all later comparisons.

3.2 Recurrent Models

Recurrent modeling is useful. In the LSTM family test CER drops from 22.63 for LSTM-1 to 21.12 for LSTM-2 and then to 16.49 for LSTM-3 at 40 epochs. At 150 epochs, LSTM-1, LSTM-2 and LSTM-3 improve to 19.67, 18.63 and 15.04. GRU-2 and RNN-2 also outperform Base, which confirms that adding recurrence helps. The results indicate that the choice of recurrent unit and the recurrent depth both matter.

Model	**Variant**	**40 epoch test CER**	**150 epoch test CER**
TDS-Conv + LSTM (1 Layer)	original (selected)	22.63	19.67
	bidirectional	21.55	No final adopted result recorded
TDS-Conv + LSTM (2 Layers)	original (selected)	21.12	18.63
	hidden size = num features	21.18	No final adopted result recorded
	bidirectional	No final adopted result recorded	No final adopted result recorded
TDS-Conv + LSTM (3 Layers)	original	20.27	No final adopted result recorded
	bidirectional (selected)	16.49	15.04
TDS-Conv + GRU (2 Layers)	default	21.94	18.72
TDS-Conv + Vanilla RNN (2 Layers)	default	20.89	18.98

We first attempted to screen variants using 40 epochs, and based on the results we extended only the most promising bidirectional models to 150 epochs. Notably, the 3-layer LSTM becomes substantially stronger once bidirectionality is introduced. Compared with the original unidirectional 3-layer exploration (20.27 at 40 epochs), the bidirectional 3-layer variant reduces test CER to 16.49

at 40 epochs and further to 15.04 at 150 epochs. This suggests that access to both past and future context is especially beneficial for this task.

3.3 Residual Models

The residual line gives the strongest results. Res-LSTM-3 reaches 16.60 at 40 epochs and 13.88 at 150 epochs, while Res-LSTM-4 improves further to 15.73 and 13.29.

Model	**Variant**	**40 epoch test CER**	**150 epoch test CER**
Res-TDS-Conv + LSTM (3 Layers)	original residual run	19.26	16.40
	bidirectional residual run	16.92	No final adopted result recorded
	dropout at end (selected)	16.60	13.88
Res-TDS-Conv + LSTM (4 Layers)	original (selected)	15.73	13.29
Res-TDS-Conv + LSTM (5 Layers)	original	20.45	No final adopted result recorded

We treated the 5-layer residual model as a 40-epoch exploratory run. However, since the result showed CER increasing rather than decreasing, we did not extend its training duration to 150 epochs. This result suggests that the gain comes from effective fusion and controlled model capacity, not from blindly increasing depth.

3.4 Attention Variant

Model	**40 epoch test CER**	**150 epoch test CER**
Res-TDS-Conv + LSTM + Local Attention	17.66	17.85

The local-attention variant improves over Base, but it does not surpass the residual LSTM. In the current data and tuning range, it is better treated as a supplementary architectural experiment than as a main result line. In addition, our attention implementation is better interpreted as a local cross-attention design between the recurrent branch and the original TDS features, rather than pure self-attention. Although this cross-branch interaction improved over the baseline, it still did not surpass the simpler residual LSTM architecture under the current training setting.

3.5 Summary Comparison

The table below summarizes the representative configuration finally adopted for each family under the two training budgets.

Model Family	40-epoch Representative Config	40 epoch test CER	150-epoch Representative Config	150 epoch test CER
Official TDS Baseline	Base (original)	23.99	Base (original)	22.41
TDS-Conv + LSTM family	LSTM-3 (bidirectional)	16.49	LSTM-3 (bidirectional)	15.04
TDS-Conv + GRU family	GRU-2 (original)	21.94	GRU-2 (original)	18.72
TDS-Conv + Vanilla RNN family	RNN-2 (original)	20.89	RNN-2 (original)	18.98
Res-TDS-Conv + LSTM family	Res-LSTM-4 (original)	15.73	Res-LSTM-4 (original)	13.29
Res-TDS-Conv + LSTM + Local Attention	Res-LSTM-Attn (original)	17.66	Res-LSTM-Attn (original)	17.85

At 40 epochs, the ranking is clear. Among the original models, Res-LSTM-4 performs best at 15.73, followed by Res-LSTM-Attn at 17.66, RNN-2 at 20.89, GRU-2 at 21.94, and Base at 23.99. Although LSTM-3 reaches 16.49 after introducing bidirectionality and outperforms most models, it still remains slightly weaker than Res-LSTM-4 (original). This indicates that the most effective path to improvement is not simply replacing recurrent units, but building a stronger recurrent structure and then integrating residual fusion. Compared with Base, Res-LSTM-4 reduces test CER by 8.26 points.

At 150 epochs, the structural gap becomes even larger. Base reaches 22.41, Res-LSTM-Attn reaches 17.85, RNN-2 reaches 18.98, LSTM-3 reaches 15.04, and Res-LSTM-4 (original) further improves to 13.29, still stronger than the bidirectional LSTM-3. Compared with Base, Res-LSTM-4 reduces test CER by 9.12 points.

Notably, although GRU-2 performs better than the baseline, its gain remains limited, and its performance is still clearly weaker than that of the bidirectional LSTM variant. This indicates that extending training does not benefit all models equally; stronger architectures gain much more from longer training than weaker ones.

3.6 Comprehensive Cross-Model Comparison

Taken together, we have reached the following three conclusions. :

- First, recurrent modeling itself is effective, because GRU-2, RNN-2, and the LSTM family all outperform Base.
- Second, among the recurrent variants we tested, LSTM performs best on this task. This suggests that a recurrent architecture with stronger long-range memory and more stable sequence modeling is better suited to noisy sEMG decoding than either vanilla RNNs or the lighter GRU variant in our setting.
- Third, Retaining local patterns within the TDS branch and fusing them with the recurrent branch is more effective than relying solely on the recurrent backend. The results from Res-LSTM-4 strongly support this view.

4 Discussion

4.1 RNN Architecture

In our research, after 150 epochs of training, the baseline model achieved a CER of approximately 22.41. A key finding of the experiments was that the introduction of explicit sequence modeling significantly improved predictive performance. The CTC framework directly predicts character sequences from unsegmented input signals and learns the temporal alignment relationships between the input sequence and output labels during the training process [2]. Since EMG signals are distributed across multiple time steps, the model must capture long-range temporal dependencies to correctly decode keystrokes. Our results confirm that, compared to a purely convolutional baseline model, adding recurrent layers after the convolutional encoder consistently reduces the CER. For instance, the

TDS-Conv + LSTM model achieved a CER of 15.04, while the Vanilla RNN model achieved a CER of 18.98; both outperform the baseline model. These results suggest that, although the TDS encoder is capable of capturing useful local patterns within EMG signals, additional recurrent modeling is still required to capture broader temporal context. Among the recurrent architectures tested in this study, the LSTM performed the best. Both the GRU and Vanilla RNN models underperformed compared to the LSTM, indicating that the gating mechanisms within the LSTM provide superior stability when modeling noisy EMG signals.

LSTM networks incorporate gating mechanisms that allow the model to selectively retain and update long-term information—a feature particularly beneficial for sequential signals exhibiting temporal dependencies [5]. In the context of EMG keystroke decoding, muscle activation patterns may span multiple time steps before a character output is generated. Consequently, modeling long-term memory is crucial for accurate prediction. Another significant finding was the advantage conferred by bidirectional sequence modeling. By adapting the LSTM layers into a bidirectional architecture, the model is able to leverage contextual information from both past and future time steps. This enhancement resulted in a substantial performance improvement: the Bidirectional LSTM model achieved a CER of 16.49, compared to 20.27 for the standard LSTM configuration. This improvement demonstrates that keystroke decoding benefits from the incorporation of additional contextual information.

Our experiments also revealed the complementary roles of convolutional and recurrent modeling. The convolutional TDS encoder is capable of effectively extracting local temporal patterns from electromyographic signals. However, these local patterns alone are insufficient for reliable decoding, as the temporal relationships between signals often span longer time windows. Recurrent layers provide an effective mechanism for capturing these long-range temporal dependencies. Consequently, the combination of convolutional feature extraction and recurrent sequence modeling enables the model to represent both the local and global temporal structures within electromyographic signals.

4.2 Residual TDS-Conv + LSTM Architecture

The most effective architecture explored in this project was the Residual TDS-Conv + LSTM model. In deep learning models, particularly as the number of layers increases. When RNN structures are employed, deep neural networks are susceptible to the vanishing gradient problem, where gradients shrink during backpropagation due to repeated multiplication of small derivatives across many layers. This phenomenon can cause parameter updates in the lower layers to slow down or even stagnate. The introduction of the residual structure was not merely a rote application of the ResNet architecture but rather drew upon the core philosophy of residual learning: by directly mapping input information to subsequent layers, it establishes a high-speed and lossless pathway for gradient backpropagation. This effectively mitigates the degradation problem inherent in deep networks, enabling the model to traverse the "valleys" of feature transformation and converge more smoothly within the parameter space. In our experiments, this residual-fusion architecture demonstrated exceptional robustness. As the training epochs progressed, the loss function curve successfully navigated past the initial "peaks" of fluctuation and rapidly entered a downward trajectory. Ultimately, after 150 epochs, the CER reached an optimal performance of 13.29. This powerfully validates the premise that while preserving local convolutional features, incorporating long-range temporal modeling enhanced by residual connections can significantly boost the model's decoding accuracy for complex sEMG signals.

In a standard CNN-RNN pipeline, convolutional features must pass entirely through the recurrent layers before reaching the classifier, a process that may distort or obscure useful local information. The residual fusion structure enables the model to maintain a direct path from the convolutional features to the output, while simultaneously incorporating temporal contextual information via a recurrent branch. This dual-path design allows the model to integrate strong local signal features with long-range contextual information—a key factor contributing to the significant reduction in CER.

4.3 Residual TDS-Conv + LSTM + Attention Architecture

We also explored the integration of a local attention mechanism atop the residual LSTM architecture. Theoretically, self-attention mechanisms allow a model to dynamically focus on important temporal regions by computing interactions between queries, keys, and values. However, global attention

requires computing pairwise interactions between all time steps, resulting in quadratic computational complexity with respect to sequence length. For long sEMG sequences, this can lead to high memory consumption and computational cost. To mitigate this issue, we adopt a local attention mechanism that restricts attention computation within fixed-size temporal windows. As both sequence length and feature dimensionality increase in tandem, the computational complexity grows quadratically, leading to a curse of dimensionality and memory overload. To address this issue, we implemented a window, a constraining mechanism within our code. Instead of performing a global scan across the entire sequence, this module partitions the long sequence into several fixed and local temporal windows. Attention weights are computed independently within each window, and the results are subsequently concatenated and merged. Although this design significantly optimized computational efficiency from an engineering standpoint, the attention-based architecture ultimately failed to outperform the standard residual LSTM model in the context of this specific task. After 150 epochs, the residual TDS-Conv + LSTM + Attention model yielded a CER of 17.85, whereas the residual TDS-Conv + LSTM model achieved a notably lower CER of 13.29. Our analysis suggests that sEMG signals exhibit a strong temporal dependency. Segmenting the signal into local windows may, to some extent, disrupt its long-range continuity. Consequently, its generalization capability is inferior to that of the Residual LSTM architecture, which facilitates the smooth propagation of information.

One reason why the addition of a local attention mechanism failed to outperform the Residual LSTM model is that the recurrent architecture is already capable of effectively capturing temporal dependencies. The additional attention mechanism may introduce unnecessary model complexity without providing significant additional information. Furthermore, restricting attention to a local window may limit the model's ability to capture long-range dependencies. A task that Bidirectional LSTMs are already well-equipped to handle. Furthermore, the attention module we implemented is not a purely self-attention mechanism. Instead, it functions as a local cross-attention module, wherein the recurrent features attend to the original TDS features. We adopted this design because, in this specific context, directly applying a local self-attention mechanism yields suboptimal results; conversely, cross-branch interaction offers a more explicit means of integrating sequential context with the preserved local convolutional structure.

4.4 Model Depth

Finally, we investigated the impact of model depth. While increasing the number of recurrent layers can enhance performance within a certain range, deeper models do not invariably yield superior results. For instance, expanding the Residual LSTM architecture from three layers to four further reduced the CER to 13.29. However, increasing the depth to five layers led to a significant decline in performance. This observation aligns with existing findings indicating that, when training data is limited, deeper networks may introduce optimization challenges or lead to overfitting [4]. Overall, our experiments demonstrate that integrating convolutional feature extraction with effective sequence modeling is crucial for sEMG-based keystroke decoding. Among the architectures explored in this project, the Residual LSTM model achieved the optimal balance between model complexity and predictive performance.

References

[1] Viswanath Sivakumar, Jeffrey Seely, Alan Du, Sean R. Bittner, Adam Berenzweig, Anuoluwapo Bolarinwa, Alexandre Gramfort, and Michael I. Mandel. emg2qwerty: A Large Dataset with Baselines for Touch Typing using Surface Electromyography. *arXiv preprint arXiv:2410.20081*, 2024. URL `https://arxiv.org/abs/2410.20081`.

[2] Alex Graves, Santiago Fernandez, Faustino Gomez, and Jurgen Schmidhuber. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 369–376, 2006. URL `https://doi.org/10.1145/1143844.1143891`.

[3] Calvin Pang et al. emg2qwerty GitHub repository. `https://github.com/Calvin-Pang/emg2qwerty`

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017. URL `https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html`.

[5] Sepp Hochreiter and Jurgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 1997. URL `https://doi.org/10.1162/neco.1997.9.8.1735`.